

Plano de Disciplina

Curso

Ciência da Computação

Fundamentação Legal

Autorizado pela Portaria n.º 264, de 03-03-2000. Publicada no DOU n.º 47, seção 1, pág 4 de 09-03-2000. Reconhecido pela Portaria n.º 1.636, de 03-06-2004. Publicada no DOU n.º 108, seção 1, pág 24 de 07-06-2004. Renovado pela Portaria n.º 348, de 09-07-2026. Publicada no DOU n.º 128, seção 1, pág 93 de 10-07-2026

Disciplina	Ano	Grade	Período
Linguagens de Programação	2026 - 2	Ciência da Computação 2024/1 202310 (Matutino)	6
C.H. Teórica	C.H. Prática	C.H. Total	
80	0	80	

Ementa

Definição e estudo dos conceitos e características das linguagens de programação. Análise dos paradigmas das linguagens de programação. Implementações práticas de programas para um variado conjunto de domínios de problemas.

Objetivos Gerais

Ao final do curso os estudantes deverão ser capazes de: compreender como utilizar os diferentes paradigmas e construtos das linguagens de programação para projetar e criar programas corretos e elegantes; comparar os diferentes paradigmas, apontando vantagens e desvantagens de cada um; escolher qual paradigma e linguagem são mais adequados para a resolução de um problema computacional específico; explicar em profundidade as características da sintaxe e semântica das linguagens; saber quando e como projetar uma pequena linguagem; aprender novas linguagens rapidamente

Competências e Habilidades:

Habilidades Gerais:

Identificar problemas que tenham solução algorítmica e desenvolver soluções para situações-problema; Gerir a sua própria aprendizagem e desenvolvimento, incluindo a gestão de tempo e competências organizacionais; Adequar-se rapidamente às mudanças tecnológicas e aos novos ambientes de trabalho; Ler e interpretar textos técnicos na língua inglesa; Extrair conclusões por indução e/ou dedução; Estabelecer relações, comparações e contrastes em diferentes situações; Resolver problemas utilizando ambientes de programação.

Habilidades Específicas:

Compreender os fatos essenciais, os conceitos, os princípios e as teorias relacionadas à Ciência da Computação para o desenvolvimento de software e hardware e suas aplicações; Reconhecer a importância do pensamento computacional no cotidiano e sua aplicação em circunstâncias apropriadas e em domínios diversos, estabelecendo relações, comparações e contrastes em diferentes situações; Especificar, projetar, implementar, manter e avaliar sistemas de computação, empregando teorias, práticas e ferramentas adequadas; E6 - Conceber soluções computacionais a partir de decisões visando o equilíbrio de todos os fatores envolvidos, estabelecendo relações; Analisar quanto um sistema baseado em computadores atende os critérios definidos para seu uso corrente e futuro (adequabilidade); Aplicar temas e princípios recorrentes, como abstração, complexidade, extrair conclusões por indução e/ou dedução, princípio de localidade de referência (caching), compartilhamento de recursos, segurança, concorrência, evolução de sistemas, entre outros, e reconhecer que esses temas e princípios são fundamentais à área de Ciência da Computação.

Contribuição para a Formação Profissional:

A disciplina de Linguagens de Programação contribui para a formação dos alunos ao capacitá-los a entender, comparar e aplicar diferentes paradigmas e construções das linguagens de programação no desenvolvimento de soluções corretas, eficientes e elegantes. Ao explorar aspectos relacionados à sintaxe e semântica, os estudantes ampliam sua habilidade analítica e crítica, tornando-se aptos a selecionar a linguagem e o paradigma mais apropriados para cada contexto computacional. Além disso, a disciplina desenvolve competências essenciais para a rápida assimilação de novas linguagens e oferece uma base sólida para que os alunos possam, quando necessário, projetar pequenas linguagens voltadas a necessidades específicas.

Abordagens voltadas para Sustentabilidade (Ambiental, Social e Econômica):

Conteúdos

DE APRENDIZAGEM

- Unidade 0: Apresentação
- Unidade 1: Sintaxe e semântica
- Unidade 2: Identificadores
- Unidade 3: Funções de 1ª Classe e de 1ª Ordem
- Unidade 4: Cálculo lambda e funções recursivas
- Unidade 5: Variáveis e estruturas mutáveis
- Unidade 6: Garbage Collection e Lazy Evalation
- Unidade 7: Continuações
- Unidade 8: Tipos
- Unidade 9: Tipos (continuação)
- Unidade 10: Inferência, união e checagem de tipos
- Unidade 11: Programação funcional
- Unidade 12: Programação lógica
- Unidade 13: Programação Orientada à Objeto
- Unidade 14: Seminários especiais

Metodologia

A disciplina será desenvolvida por meio de uma abordagem teórico-prática, combinando exposição dialogada, análise comparativa de linguagens e paradigmas, atividades de pesquisa e implementação de programas.

As aulas teóricas serão conduzidas de forma expositiva e dialogada, com apresentação dos conceitos fundamentais de linguagens de programação, seguida de discussões em sala que estimulem a comparação entre diferentes paradigmas e características de linguagens. Sempre que possível, serão utilizados exemplos concretos em C++ e em outras linguagens de referência (como Haskell, Prolog, Python ou Java), a fim de evidenciar diferenças e similaridades conceituais.

As atividades práticas ocorrerão em laboratório, com a implementação de programas em C++ voltados a domínios variados de problemas, permitindo aos alunos vivenciar na prática os conceitos estudados (sintaxe e semântica, tipagem, gerenciamento de memória, funções de primeira classe, programação funcional, orientação a objetos, entre outros). Quando o ambiente de laboratório ainda não dispuser do compilador, serão propostas atividades alternativas de pesquisa, análise de código e comparação entre linguagens, de modo a manter a continuidade do processo de aprendizagem.

A Unidade 14 (Seminários especiais) será reservada à apresentação de trabalhos de pesquisa elaborados pelos alunos, promovendo a socialização do conhecimento e o aprofundamento de temas específicos da disciplina.

A avaliação será processual e diversificada, contemplando participação em discussões, trabalhos de pesquisa, exercícios práticos e, quando aplicável, provas escritas, priorizando a compreensão conceitual e a capacidade de análise crítica dos alunos.

Práticas Emergentes e/ou Inovadoras

Dentre as práticas previstas, destacam-se:

Aulas invertidas seletivas: em unidades de maior densidade conceitual (como Cálculo Lambda, Continuações e Sistemas de Tipos), os alunos terão acesso prévio a materiais introdutórios (vídeos curtos, textos ou exemplos interativos), reservando o tempo de aula para discussão, resolução de problemas e análise comparativa entre linguagens.

Uso de ambientes online de execução e visualização: enquanto o compilador C++ não estiver disponível nos laboratórios, serão utilizados recursos como Compiler Explorer (Godbolt), onlinegdb, Replit e Jupyter notebooks (quando aplicável), permitindo experimentação imediata de código e observação de aspectos de sintaxe, tipagem e avaliação.

Análise comparativa mediada por pares: os alunos, organizados em duplas ou pequenos grupos, realizarão investigações sobre como diferentes linguagens implementam um mesmo conceito (ex.: gerenciamento de memória, funções de primeira classe, tipagem estática vs. dinâmica), apresentando os resultados em formato de seminário ou relatório colaborativo.

Seminários especiais com protagonismo discente: a Unidade 14 será conduzida pelos próprios alunos, que escolherão temas de aprofundamento relacionados à ementa e mediarão a discussão, promovendo autonomia e socialização do conhecimento.

Integração crítica de ferramentas digitais e inteligência artificial: será incentivado o uso responsável de assistentes de código e ferramentas de análise (como ChatGPT, Claude ou GitHub Copilot) para explorar e questionar implementações, sempre com a exigência de validação conceitual e justificativa teórica por parte do aluno.

Essas práticas visam aproximar o conteúdo teórico da experimentação concreta, desenvolver o pensamento crítico sobre o projeto de linguagens e preparar os estudantes para a diversidade de paradigmas encontrados no mercado e na pesquisa.

Avaliação de Aprendizagem

A avaliação da aprendizagem será realizada de forma contínua e processual, em conformidade com as orientações institucionais vigentes sobre a aplicação de trabalhos e provas, contemplando tanto a dimensão teórica quanto a prática da disciplina. Serão utilizados os seguintes instrumentos de avaliação:

Provas escritas (teóricas e/ou teórico-práticas), aplicadas em datas previamente definidas no calendário acadêmico, visando verificar a compreensão dos conceitos, paradigmas e características das linguagens de programação.

Trabalhos de pesquisa e análise, individuais ou em dupla, com ênfase na comparação entre linguagens e no aprofundamento de unidades específicas da ementa (incluindo o trabalho “C++ sob a ótica dos conceitos de Linguagens de Programação”).

Atividades práticas e exercícios, realizados em laboratório ou de forma assíncrona, quando o ambiente de desenvolvimento estiver disponível.

Seminários e apresentações (Unidade 14), com avaliação da clareza, profundidade conceitual e capacidade de mediação da discussão.

A composição da nota final observará a ponderação definida pela instituição, assegurando o equilíbrio entre avaliações individuais (provas) e atividades de natureza formativa e colaborativa (trabalhos e seminários).

Os critérios específicos de correção de cada instrumento serão divulgados antecipadamente, priorizando a clareza conceitual, a profundidade da análise, a capacidade de comparação entre paradigmas/linguagens e a qualidade das referências e exemplos utilizados.A frequência mínima exigida e as regras de recuperação (quando aplicáveis) seguirão integralmente o regimento institucional.

Bibliografia Básica

ABELSON, Harold; SUSSMAN, Gerald Jay; SUSSMAN, Julie. Structure and interpretation of computer programs. 2. ed. Cambridge: MIT Press, 1997 e edições anteriores.

SEBESTA, ROBERT W. Conceitos de linguagens de programação. 11. ed. Porto Alegre, RS: Bookman, 2018 e edições anteriores.

VAREJÃO, FLÁVIO MIGUEL. Linguagens de programação: Java, C e C++ e outras: conceitos e técnicas. Rio de Janeiro: Elsevier, 2004.

Bibliografia Complementar

CORMEN, Thomas H et al. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro, RJ: LTC, 2022.

KNUTH, Donald E. The art of computer programming. 2. ed. Reading: Addison-Wesley, 1998 e edições anteriores.

MITCHELL, JOHN C. Foundations for programming languages. Cambridge: MIT Press, 1998 e edições anteriores.

SANTOS, Rafael. Introdução à programação orientada a objetos usando Java. Rio de Janeiro: Campus, 2013 e edições anteriores.

SINTES, TONY. Aprenda programação orientada a objetos em 21 dias. São Paulo, SP: Pearson, 2002